

.NET Microservices Complete Tutorial for Beginners (Production-Oriented Guide)

What Are Microservices?

Microservices are an architectural style where a large application is split into multiple small, independent services.

Instead of building one large monolithic application:

E-Commerce Application

- Product Module
- Order Module
- Customer Module
- Payment Module
- Inventory Module

You create separate services:

Product Service
Order Service
Customer Service
Payment Service
Inventory Service

Each service:

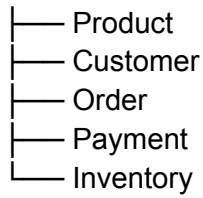
- Has its own database
- Can be developed independently
- Can be deployed independently
- Can scale independently

Why Microservices?

Monolithic Architecture Problems

Application

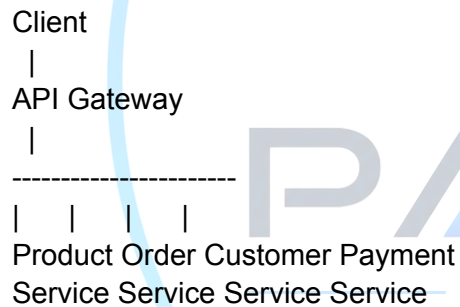
|



Problems:

- ✗ Difficult to maintain
- ✗ Slow deployments
- ✗ Entire application redeployed for small change
- ✗ Difficult scaling
- ✗ Technology lock-in

Microservices Architecture



Benefits:

- ✓ Independent deployment
- ✓ Better scalability
- ✓ Easier maintenance
- ✓ Team autonomy
- ✓ Technology flexibility

Real-World Example

Amazon

Microservices:

- Product Service
- Cart Service
- Order Service
- Inventory Service
- Payment Service
- Notification Service

Each service works independently.

Technologies Used in .NET Microservices

Backend

- ASP.NET Core 8
- C#
- Entity Framework Core
- SQL Server

Communication

- REST API
- gRPC
- RabbitMQ
- Azure Service Bus

Authentication

- JWT
- Identity Server

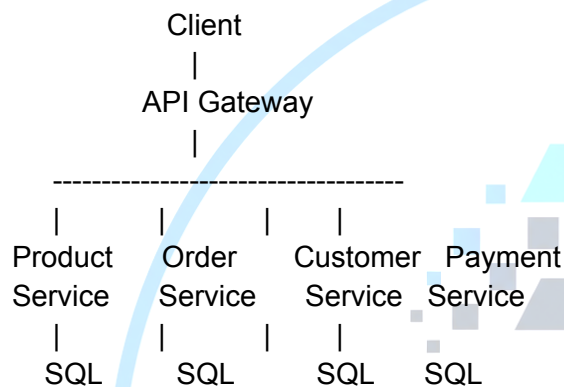
Containerization

- Docker
- Kubernetes

Monitoring

- Serilog
 - ELK
 - Prometheus
 - Grafana
-

Microservices Architecture Diagram



Step 1: Create Product Microservice

Create Project

```
dotnet new webapi -n ProductService
```

Run:

```
dotnet run
```

Swagger opens:

```
https://localhost:5001/swagger
```

Product Model

```
public class Product
{
    public int Id { get; set; }
```

```
public string Name { get; set; }

public decimal Price { get; set; }

public int Stock { get; set; }
}
```

Product DbContext

```
using Microsoft.EntityFrameworkCore;

public class ProductDbContext : DbContext
{
    public ProductDbContext(
        DbContextOptions<ProductDbContext> options)
        : base(options)
    {
    }

    public DbSet<Product> Products { get; set; }
}
```

Install EF Core

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
```

```
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

Connection String

appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection":
      "Server=.;Database=ProductDB;Trusted_Connection=True;TrustServerCertificate=True"
  }
}
```

```
}  
}
```

Register Services

Program.cs

```
builder.Services.AddDbContext<ProductDbContext>(  
options =>  
options.UseSqlServer(  
builder.Configuration.GetConnectionString("DefaultConnection")));
```

Create Controller

```
[ApiController]  
[Route("api/[controller]")]  
public class ProductsController : ControllerBase  
{  
    private readonly ProductDbContext _context;  
  
    public ProductsController(ProductDbContext context)  
    {  
        _context = context;  
    }  
  
    [HttpGet]  
    public async Task<IActionResult> Get()  
    {  
        return Ok(await _context.Products.ToListAsync());  
    }  
  
    [HttpPost]  
    public async Task<IActionResult> Create(Product product)  
    {  
        _context.Products.Add(product);  
  
        await _context.SaveChangesAsync();  
  
        return Ok(product);  
    }  
}
```

```
}
```

Create Migration

```
dotnet ef migrations add InitialCreate
```

```
dotnet ef database update
```

Database created.

Step 2: Create Order Service

```
dotnet new webapi -n OrderService
```

Order Model:

```
public class Order
{
    public int Id { get; set; }

    public int ProductId { get; set; }

    public int Quantity { get; set; }

    public DateTime OrderDate { get; set; }
}
```

Service-to-Service Communication

Order Service needs Product Service.

REST API Call

```
public class ProductClient
{
    private readonly HttpClient _httpClient;

    public ProductClient(HttpClient httpClient)
```

```

{
    _httpClient = httpClient;
}

public async Task<ProductDto> GetProduct(int id)
{
    return await _httpClient
        .GetFromJsonAsync<ProductDto>(
            $"api/products/{id}");
}
}

```

Register:

```

builder.Services.AddHttpClient<ProductClient>(
    client =>
    {
        client.BaseAddress =
            new Uri("https://localhost:7001");
    });

```

DTO Pattern

Never expose Entity directly.

```

public class ProductDto
{
    public int Id { get; set; }

    public string Name { get; set; }

    public decimal Price { get; set; }
}

```

Repository Pattern

Interface

```

public interface IProductRepository
{

```



```
Task<IEnumerable<Product>> GetAllAsync();

Task<Product> GetByIdAsync(int id);

Task AddAsync(Product product);
}
```

Implementation

```
public class ProductRepository
    : IProductRepository
{
    private readonly ProductDbContext _context;

    public ProductRepository(ProductDbContext context)
    {
        _context = context;
    }

    public async Task<IEnumerable<Product>>
        GetAllAsync()
    {
        return await _context.Products.ToListAsync();
    }

    public async Task<Product>
        GetByIdAsync(int id)
    {
        return await _context.Products.FindAsync(id);
    }

    public async Task AddAsync(Product product)
    {
        await _context.Products.AddAsync(product);

        await _context.SaveChangesAsync();
    }
}
```

CQRS Pattern

Separate Reads and Writes.

Command

```
public record CreateProductCommand(  
    string Name,  
    decimal Price,  
    int Stock);
```

Query

```
public record GetProductsQuery();
```

Benefits:

- Better scalability
- Better performance
- Easier maintenance

MediatR Integration

Install:

```
dotnet add package MediatR
```

Command Handler

```
public class CreateProductHandler  
:  
    IRequestHandler<CreateProductCommand,int>  
{  
    public async Task<int>  
        Handle(CreateProductCommand request,  
            CancellationToken cancellationToken)  
        {  
            return 1;  
        }  
}
```

API Gateway

Popular Options:

- Ocelot
- YARP

Install:

```
dotnet add package Yarp.ReverseProxy
```

Program.cs

```
builder.Services  
.AddReverseProxy()  
.LoadFromConfig(  
builder.Configuration.GetSection("ReverseProxy"));
```

appsettings.json

```
{  
  "ReverseProxy": {  
    "Routes": {  
      "productRoute": {  
        "ClusterId": "productCluster",  
        "Match": {  
          "Path": "/products/{**catch-all}"  
        }  
      }  
    }  
  }  
}
```

JWT Authentication

Install

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

Configure

```
builder.Services  
.AddAuthentication(  
JwtBearerDefaults.AuthenticationScheme)  
.AddJwtBearer(options =>  
{  
  options.TokenValidationParameters =
```

```
new TokenValidationParameters
{
    ValidateIssuer = true,
    ValidateAudience = true,
    ValidateLifetime = true
};
});
```

Controller

```
[Authorize]
[HttpGet]
public IActionResult Get()
{
    return Ok();
}
```

RabbitMQ Messaging

Install:

```
dotnet add package RabbitMQ.Client
```

Producer

```
var factory =
new ConnectionFactory()
{
    HostName = "localhost"
};
```

```
using var connection =
factory.CreateConnection();
```

```
using var channel =
connection.CreateModel();
```

```
channel.QueueDeclare(
"orders",
false,
false,
false);
```

```
var body =
Encoding.UTF8.GetBytes("New Order");

channel.BasicPublish(
    "",
    "orders",
    null,
    body);

Consumer

var consumer =
new EventingBasicConsumer(channel);

consumer.Received += (model, ea) =>
{
    var message =
        Encoding.UTF8.GetString(ea.Body.ToArray());

    Console.WriteLine(message);
};
```

Dockerizing Microservices

Dockerfile

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0
```

```
WORKDIR /app
```

```
COPY . .
```

```
ENTRYPOINT ["dotnet", "ProductService.dll"]
```

Build

```
docker build -t productservice .
```

Run

```
docker run -d -p 8080:80 productservice
```

Docker Compose

version: '3.8'

services:

productservice:

build: .

ports:

- "5001:80"

orderservice:

build: .

ports:

- "5002:80"

sqlserver:

image: mcr.microsoft.com/mssql/server

Run:

docker-compose up -d

Kubernetes Basics

Deployment

apiVersion: apps/v1

kind: Deployment

metadata:

name: productservice

Apply

kubectl apply -f deployment.yaml

Logging Using Serilog

Install

```
dotnet add package Serilog.AspNetCore
```

Program.cs

```
builder.Host.UseSerilog((ctx, lc) =>
{
    lc.WriteTo.Console();
});
```

Usage

```
_logger.LogInformation(
"Product Created");
```

Health Checks

```
builder.Services
.AddHealthChecks()
.AddSqlServer(connectionString);
```

Map

```
app.MapHealthChecks("/health");
```

Production Best Practices

Use Clean Architecture

API
Application
Domain
Infrastructure

Use CQRS

Commands
Queries

Use Messaging

RabbitMQ
Kafka
Azure Service Bus

Use API Gateway

YARP
Ocelot

Use Containerization

Docker
Kubernetes

Use Monitoring

Grafana
Prometheus
ELK

Use Authentication

JWT
OAuth2
OpenID Connect

